

LAPORAN TUGAS 6

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Jhodi Andika (2410651050)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

BAB 1

PENDAHULUAN

1.1 Tujuan

Praktikum ini memberikan pemahaman mendalam tentang bagaimana deadlock dapat muncul dan mengganggu kerja sistem operasi. Melalui serangkaian contoh program C, kita menelusuri bagaimana proses dapat saling menahan resource hingga membentuk siklus yang tidak berujung. Tidak hanya memahami konsepnya, kita juga melihat langsung bagaimana mekanisme mutual exclusion menjadi pemicu utama konflik antar proses. Praktikum ini turut menguji solusi praktis seperti penggunaan timeout untuk memutus kebuntuan, penerapan lock ordering agar proses mengambil kunci secara teratur, serta Banker's Algorithm yang menjaga kestabilan sistem. Setelah menyelesaikan seluruh bagian, kita memperoleh pemahaman yang kuat tentang cara mencegah proses saling menghambat dalam lingkungan konkuren..

BAB 2

PEMBAHASAN

2.1 Analisis Deadlock

Jalankan program `deadlock_simple.c`

2.1.1 Mengapa program mengalami deadlock

Deadlock muncul ketika dua thread terjebak dalam ketergantungan siklik saat mencoba mengakses resource yang saling terkunci. Thread pertama sudah memegang lock1 lalu menunggu lock2, sedangkan thread kedua telah menguasai lock2 dan menunggu lock1. Pola saling menunggu ini menghasilkan kondisi mandek karena masing-masing thread tidak dapat melanjutkan eksekusi tanpa resource yang sedang dikuasai thread lain. Tidak ada pihak yang dapat bergerak ataupun melepaskan kunci terlebih dahulu, sehingga keduanya berhenti dalam keadaan blok permanen. Pada akhirnya, seluruh proses program macet dan tidak dapat menyelesaikan eksekusi.

2.1.2 Kondisi deadlock apa saja yang terpenuhi

Terdapat 4 kondisi deadlock yang terpenuhi yakni:

1. Mutual Exclusion:

Hanya ada satu proses yang dapat menggunakan suatu sumber daya pada satu waktu. Sumber daya tersebut tidak dapat dibagi.

2. Hold and Wait:

Sebuah proses yang sudah memegang setidaknya satu sumber daya, menunggu untuk mendapatkan sumber daya tambahan yang sedang dipegang oleh proses lain.

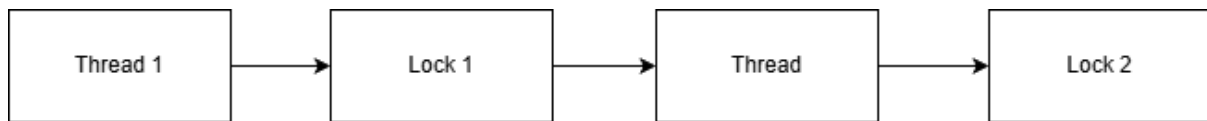
3. No Preemption:

Sumber daya yang telah dialokasikan kepada suatu proses tidak dapat diambil paksa darinya. Proses tersebut harus melepaskannya secara sukarela setelah selesai digunakan.

4. Circular Wait:

Terdapat rangkaian proses menunggu yang melingkar (siklus), di mana setiap proses menunggu sumber daya yang dipegang oleh proses berikutnya dalam rantai tersebut.

2.1.3 Gambar diagram resource allocation graph-nya



2.2 Implementasi Pencegahan

Modifikasi program deadlock_simple.c

2.2.1 Lock ordering

Outputnya:

```
asus@LAPTOP-VRS206DG:~/jhodiandika$ nano deadlock_simple.c
asus@LAPTOP-VRS206DG:~/jhodiandika$ gcc deadlock_simple.c -o deadlock_simple -lpthread
asus@LAPTOP-VRS206DG:~/jhodiandika$ ./deadlock_simple
Thread 1: mencoba lock mutex1...
Thread 1: mutex1 terkunci.
Thread 1: mencoba lock mutex2...
Thread 1: mutex2 terkunci.
Thread 1: menjalankan critical section...
Thread 2: mencoba lock mutex1...
Thread 1: selesai.
Thread 2: mutex1 terkunci.
Thread 2: mencoba lock mutex2...
Thread 2: mutex2 terkunci.
Thread 2: menjalankan critical section...
Thread 2: selesai.
asus@LAPTOP-VRS206DG:~/jhodiandika$
```

Kode:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t mutex1;
pthread_mutex_t mutex2;

void* thread1(void* arg) {
    printf("Thread 1: mencoba lock mutex1...\n");
    pthread_mutex_lock(&mutex1);
    printf("Thread 1: mutex1 terkunci.\n");

    printf("Thread 1: mencoba lock mutex2...\n");
```

```

pthread_mutex_lock(&mutex2);
printf("Thread 1: mutex2 terkunci.\n");
printf("Thread 1: menjalankan critical section...\n");
sleep(1);

pthread_mutex_unlock(&mutex2);
pthread_mutex_unlock(&mutex1);
printf("Thread 1: selesai.\n");

return NULL;
}

void* thread2(void* arg) {
    printf("Thread 2: mencoba lock mutex1...\n");
    pthread_mutex_lock(&mutex1);
    printf("Thread 2: mutex1 terkunci.\n");

    printf("Thread 2: mencoba lock mutex2...\n");
    pthread_mutex_lock(&mutex2);
    printf("Thread 2: mutex2 terkunci.\n");

    printf("Thread 2: menjalankan critical section...\n");
    sleep(1);

    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);
    printf("Thread 2: selesai.\n");
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&mutex1, NULL);
    pthread_mutex_init(&mutex2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);

```

```

pthread_create(&t2, NULL, thread2, NULL);

pthread_join(t1, NULL);
pthread_join(t2, NULL);

pthread_mutex_destroy(&mutex1);
pthread_mutex_destroy(&mutex2);

return 0;
}

```

2.2.2 Try-lock dengan retry

Outputnya:

```

asus@LAPTOP-VRS206DG:~/jhodiandika$ gcc deadlock_simple_try_lock.c -o deadlock_simple_try_lock -lpthread
asus@LAPTOP-VRS206DG:~/jhodiandika$ ./deadlock_simple_try_lock
Thread 1: mencoba lock mutex1...
Thread 1: mencoba lock mutex2...
Thread 1: semua mutex terkunci.
Thread 1: critical section...
Thread 2: mencoba lock mutex2...
Thread 1: selesai.
Thread 2: mencoba lock mutex1...
Thread 2: semua mutex terkunci.
Thread 2: critical section...
Thread 2: selesai.
asus@LAPTOP-VRS206DG:~/jhodiandika$ |

```

Kode:

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t mutex1;
pthread_mutex_t mutex2;

void* thread1(void* arg) {
    while (1) {
        printf("Thread 1: mencoba lock mutex1...\n");
        pthread_mutex_lock(&mutex1);

        printf("Thread 1: mencoba lock mutex2...\n");
        if (pthread_mutex_trylock(&mutex2) == 0) {
            printf("Thread 1: semua mutex terkunci.\n");
            break;
        } else {
            printf("Thread 1: gagal lock mutex2, retry...\n");
            pthread_mutex_unlock(&mutex1);
            usleep(100000); // 0.1 detik
        }
    }

    printf("Thread 1: critical section...\n");
    sleep(1);
}

```

```

pthread_mutex_unlock(&mutex2);
pthread_mutex_unlock(&mutex1);
printf("Thread 1: selesai.\n");

return NULL;
}

void* thread2(void* arg) {
    while (1) {
        printf("Thread 2: mencoba lock mutex2...\n");
        pthread_mutex_lock(&mutex2);

        printf("Thread 2: mencoba lock mutex1...\n");
        if (pthread_mutex_trylock(&mutex1) == 0) {
            printf("Thread 2: semua mutex terkunci.\n");
            break;
        } else {
            printf("Thread 2: gagal lock mutex1, retry...\n");
            pthread_mutex_unlock(&mutex2);
            usleep(100000); // 0.1 detik
        }
    }
}

printf("Thread 2: critical section...\n");
sleep(1);
pthread_mutex_unlock(&mutex1);
pthread_mutex_unlock(&mutex2);
printf("Thread 2: selesai.\n");

return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&mutex1, NULL);
    pthread_mutex_init(&mutex2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&mutex1);
    pthread_mutex_destroy(&mutex2);

    return 0;
}

```

2.2.3 Bandingkan kedua metode tersebut

- Kode pertama menunjukkan deadlock klasik. Thread 1 mengunci mutex1 lalu mencoba mutex2, sementara Thread 2 mengunci mutex2 lalu mencoba mutex1. Kedua thread saling menunggu selamanya karena tidak ada yang mau melepas kunci pertama. Program akan hang permanen.
- Kode kedua menggunakan pendekatan trylock untuk mencegah deadlock. Jika thread gagal mendapatkan kunci kedua, ia akan melepas kunci pertama dan mencoba lagi setelah jeda waktu. Strategi ini memecah kebuntuan dengan memaksa thread mundur sementara. Waktu tunggu yang berbeda mencegah livelock.

2.3 Banker's algorithm

Buat program baru dengan data:

- 4 Proses
- 3 jenis resource
- Available: [5, 4, 3]
- Implementasikan request resource dan cek keamanan sistem

Kode:

```
#include <stdio.h>
#include <stdbool.h>

#define P 4 // jumlah proses
#define R 3 // jumlah jenis resource

int available[R] = {5, 4, 3};

int maximum[P][R] = {
    {4, 3, 2},
    {2, 2, 3},
    {3, 3, 2},
    {5, 3, 3}
};

int allocation[P][R] = {
    {1, 0, 1},
```



```

    {1, 1, 2},
    {1, 0, 3},
    {0, 2, 1}
};

int need[P][R];

void calcNeed() {
    int i, j;
    for (i = 0; i < P; i++) {
        for (j = 0; j < R; j++) {
            need[i][j] = maximum[i][j] - allocation[i][j];
        }
    }
}

bool isSafe() {
    bool finish[P] = {false};
    int safeSeq[P];
    int work[R];
    int i, j, k;
    int count = 0;

    for (i = 0; i < R; i++) {
        work[i] = available[i];
    }

    while (count < P) {
        bool found = false;

        for (i = 0; i < P; i++) {
            if (!finish[i]) {
                bool canRun = true;
                for (j = 0; j < R; j++) {
                    if (need[i][j] > work[j]) {
                        canRun = false;

```

```

        break;
    }
}

if (canRun) {
    for (k = 0; k < R; k++) {
        work[k] += allocation[i][k];
    }
    safeSeq[count] = i;
    finish[i] = true;
    count++;
    found = true;
}
}

if (!found) {
    return false;
}

printf("Safe Sequence: ");
for (i = 0; i < P; i++) {
    printf("%d ", safeSeq[i]);
}
printf("\n");

return true;
}

bool requestResource(int p, int req[]) {
    int i;

    printf("\nRequest dari proses P%d: ", p);
    for (i = 0; i < R; i++) {
        printf("%d ", req[i]);
    }
}

```

```

}
printf("\n");

for (i = 0; i < R; i++) {
    if (req[i] > need[p][i]) {
        printf("Request lebih besar dari need proses.\n");
        return false;
    }
}

for (i = 0; i < R; i++) {
    if (req[i] > available[i]) {
        printf("Resource tidak cukup. Request ditolak.\n");
        return false;
    }
}

for (i = 0; i < R; i++) {
    available[i] -= req[i];
    allocation[p][i] += req[i];
    need[p][i] -= req[i];
}

if (isSafe()) {
    printf("Request dapat diberikan.\n");
    printf("Sistem tetap aman.\n");
    return true;
} else {
    printf("Request menyebabkan sistem tidak aman.\n");
    printf("Rollback.\n");

    for (i = 0; i < R; i++) {
        available[i] += req[i];
        allocation[p][i] -= req[i];
        need[p][i] += req[i];
    }
}

```

```

        return false;
    }
}

int main() {

    calcNeed();

    printf("Available awal: %d %d %d\n", available[0], available[1], available[2]);

    if (isSafe()) {
        printf("Sistem aman pada kondisi awal.\n");
    } else {
        printf("Sistem tidak aman pada kondisi awal.\n");
    }

    int req1[R] = {1, 0, 1};
    requestResource(1, req1);

    int req2[R] = {2, 1, 0};
    requestResource(0, req2);

    printf("\nProgram selesai.\n");
    return 0;
}

```

Output:

```

asus@LAPTOP-VRS206DG:~$ nano barkers_algoritem.c
asus@LAPTOP-VRS206DG:~$ gcc barkers_algoritem.c -o barkers_algoritem -lpthread
asus@LAPTOP-VRS206DG:~$ ./barkers_algoritem
Available awal: 5 4 3
Safe Sequence: 0 1 2 3
Sistem aman pada kondisi awal.

Request dari proses P1: 1 0 1
Safe Sequence: 0 1 2 3
Request dapat diberikan.
Sistem tetap aman.

Request dari proses P0: 2 1 0
Safe Sequence: 0 1 2 3
Request dapat diberikan.
Sistem tetap aman.

Program selesai.
asus@LAPTOP-VRS206DG:~$ |

```

Penjelasan:

Banker's Algorithm merupakan pendekatan strategis untuk mencegah terjadinya deadlock pada sistem operasi. Algoritma ini mensimulasikan proses alokasi resource secara hati-hati dengan memastikan setiap permintaan baru tidak membawa sistem ke kondisi yang tidak aman. Setiap request dievaluasi terlebih dahulu, dan hanya akan disetujui jika tetap memungkinkan terbentuknya *safe sequence*—yaitu urutan eksekusi proses yang menjamin semuanya dapat selesai tanpa konflik. Melalui mekanisme pengecekan kondisi aman serta kemampuan melakukan rollback ketika alokasi berpotensi menimbulkan bahaya, program ini menunjukkan bagaimana algoritma tersebut mampu menjaga kestabilan dan keandalan sistem dalam lingkungan konkuren.

2.4 Real Case

Buat simulasi deadlock untuk kasus: "Dua orang ingin transfer uang antar rekening secara bersamaan"

- Orang A: transfer dari Rekening 1 ke Rekening 2
- Orang B: transfer dari Rekening 2 ke Rekening 1
- Implementasikan deadlock prevention

Output:

```
asus@LAPTOP-VRS206DG:~$ nano deadlock_prevetion.c
asus@LAPTOP-VRS206DG:~$ gcc deadlock_prevetion.c -o deadlock_prevetion -lpthread
asus@LAPTOP-VRS206DG:~$ ./deadlock_prevetion
SIMULASI TRANSFER TANPA DEADLOCK

Saldo awal:
Rek1 = 1000
Rek2 = 2000

A mulai transfer 300 dari Rek1 ke Rek2...
B mulai transfer 500 dari Rek2 ke Rek1...
A selesai transfer.
B selesai transfer.

Saldo akhir:
Rek1 = 1200
Rek2 = 1800

PROGRAM SELESAI TANPA DEADLOCK
asus@LAPTOP-VRS206DG:~$ |
```

Kode:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

int saldo1 = 1000;
int saldo2 = 2000;

pthread_mutex_t lock_rek1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock_rek2 = PTHREAD_MUTEX_INITIALIZER;

void lock_in_order(pthread_mutex_t *first, pthread_mutex_t *second) {
```

```

pthread_mutex_lock(first);
pthread_mutex_lock(second);
}

void unlock_in_order(pthread_mutex_t *first, pthread_mutex_t *second) {
    pthread_mutex_unlock(second);
    pthread_mutex_unlock(first);
}

void* transfer_A(void* arg) {
    printf("A mulai transfer 300 dari Rek1 ke Rek2...\n");

    lock_in_order(&lock_rek1, &lock_rek2);

    saldo1 -= 300;
    saldo2 += 300;
    sleep(1);

    printf("A selesai transfer.\n");

    unlock_in_order(&lock_rek1, &lock_rek2);

    return NULL;
}

void* transfer_B(void* arg) {
    printf("B mulai transfer 500 dari Rek2 ke Rek1...\n");

    lock_in_order(&lock_rek1, &lock_rek2);

    saldo2 -= 500;
    saldo1 += 500;
    sleep(1);

    printf("B selesai transfer.\n");

    unlock_in_order(&lock_rek1, &lock_rek2);

    return NULL;
}

int main() {
    pthread_t A, B;

    printf("SIMULASI TRANSFER TANPA DEADLOCK\n\n");
    printf("Saldo awal:\nRek1 = %d\nRek2 = %d\n\n", saldo1, saldo2);

    pthread_create(&A, NULL, transfer_A, NULL);
    pthread_create(&B, NULL, transfer_B, NULL);
    pthread_join(A, NULL);
    pthread_join(B, NULL);

    printf("\nSaldo akhir:\nRek1 = %d\nRek2 = %d\n", saldo1, saldo2);
    printf("\nPROGRAM SELESAI TANPA DEADLOCK\n");
    return 0;
}

```

```
}
```

Penjelasan:

Program ini memodelkan proses transfer dana antara dua rekening dengan memanfaatkan dua thread yang berjalan secara paralel. Masing-masing thread melakukan pemindahan saldo dari satu akun ke akun lainnya. Untuk menghindari potensi deadlock, program menerapkan strategi lock ordering, yaitu selalu mengunci rekening pertama terlebih dahulu sebelum akun kedua dikunci. Aturan urutan ini mencegah terbentuknya circular wait. Setelah kedua lock berhasil diamankan, program mengurangi saldo dari rekening asal dan menambahkannya pada rekening tujuan sebelum akhirnya melepaskan seluruh kunci. Pendekatan ini memastikan operasi berlangsung aman, data tetap konsisten, dan tidak ada thread yang saling menunggu secara tak berujung..

BAB 3

PENUTUP

3.1 Kesimpulan

Deadlock merupakan salah satu masalah serius dalam sistem paralel karena dapat menyebabkan seluruh proses terhenti akibat circular wait. Berdasarkan percobaan, teknik lock ordering terbukti efektif mencegah kebuntuan dengan menetapkan urutan penguncian yang konsisten. Pendekatan timeout juga dapat membantu meskipun sifatnya memaksa. Selain itu, Banker's Algorithm mampu menjaga keamanan dan kelancaran sistem melalui analisis yang cermat. Dengan penerapan strategi ini, sistem menjadi lebih stabil dan terhindar dari kemacetan total.